

A PROJECT SYNOPSIS ON

POORAM CONNECT

*A Centralized Digital Ecosystem for Kerala Temple Festival Coordination &
Resource Booking*

Submitted by:

Gupthan Vishnu Prasad	Anupama Krishnakumar
Jeffin	Ajal
Ambili Chandran	Amal
Athira	Farsheena
V Vishnu	Jifana

in partial fulfillment of the requirements for the award of the degree of

MASTER OF COMPUTER APPLICATIONS



DEPARTMENT OF COMPUTER APPLICATIONS

**NEHRU COLLEGE OF ENGINEERING AND RESEARCH
CENTRE (NCERC)**

Approved by AICTE, Affiliated to APJ Abdul Kalam Technological University
Pampady, Thiruvilwamala, Thrissur, Kerala - 680588

JULY 2026

TABLE OF CONTENTS

Chapter / Section No.	Title	Page No.
1	INTRODUCTION	3
1.1	Background of Kerala Poorams & Festivals	3
1.2	The Need for a Digital Platform	3
2	PROBLEM STATEMENT	3
2.1	Current System Realities	3
2.2	Identified Pain Points	3
3	OBJECTIVES	4
4	SCOPE OF THE PROJECT	4
4.1	Target Users	4
4.2	System Boundaries & Constraints	4
4.3	Future Horizons	4
5	PROPOSED SYSTEM	5
5.1	System Architecture Overview	5
5.2	Concurrency and Booking	5

	Logic	
5.3	Database Relational Schema	6
6	MODULES OF THE PROJECT	7
7	SOFTWARE AND HARDWARE REQUIREMENTS	8
8	EXPECTED OUTCOME	8
9	CONCLUSION	9
10	REFERENCES	9

1. INTRODUCTION

1.1 Background of Kerala Poorams & Festivals

Kerala, celebrated as 'God's Own Country', is home to an extraordinary cultural heritage. Central to this heritage are the grand temple festivals, locally known as 'Poorams' and 'Ulsavams'. These festivals, which span from simple rural celebrations to colossal spectacles like the iconic Thrissur Pooram and Nemmara-Vellangi Vela, attract millions of devotees and tourists annually. A classic Kerala Pooram is a sensory experience involving magnificent processions of caparisoned elephants (Gajarajas), traditional percussion orchestras (Pandi Melam, Panchavadyam), and fireworks.

Organizing these festivals requires coordination with multiple stakeholders. The primary resources include caparisoned elephants, which are owned by private individuals, temples, or Devaswom Boards. The majestic appeal of these elephants depends on traditional ornaments: the 'Nettipattam' (golden forehead ornaments), 'Venchamaram' (white fly whisks), and 'Aalavattom' (peacock feather fans), alongside vibrant, layered decorative umbrellas ('Kudamattom') rented out by traditional accessory owners.

1.2 The Need for a Digital Platform

Despite the massive scale and strict safety guidelines, the coordination has remained manual and fragmented. Festival committees have relied on verbal agreements, phone calls, and handwritten logs to secure resources. Without a centralized system, elephant double-bookings, stock discrepancies of rental accessories, lack of safety verifications, and public coordination failures occur frequently. 'Pooram Connect' is conceived to establish a centralized digital platform that streamlines communication, guarantees resource availability, mandates safety clearances, and updates the public with real-time festival timelines.

2. PROBLEM STATEMENT

Currently, Kerala's temple festivals are managed by independent regional devaswoms and local temple committees. These committees work in absolute silos, causing significant friction in securing resources:

- **Double Booking of Elephants:** Elephants are single-occupancy entities. Under manual frameworks, inadvertent double bookings occur, causing immense pressure on committees and leading to legal and financial disputes.

- **Accessory Inventory Over-Allocation:** Traditional ornaments are rented in bulk. Owners rely on manual stock logs, which leads to over-booking during the peak festival season, causing last-minute shortages during critical rituals.
- **Regulatory Non-Compliance:** Government regulations require all elephants to possess active veterinary fitness certificates. Verification is currently manual and slow, which allows unverified animals into crowded arenas, posing safety hazards.

3. OBJECTIVES

The primary goal of 'Pooram Connect' is to centralize Kerala's festival management workflow. The project is focused on achieving the following clear objectives:

- 1. Centralization of Resources:** Build a comprehensive online directory of verified elephants, traditional accessories, and temple festival schedules, making detailed profiles accessible state-wide.
- 2. Concurrency Checks:** Develop an algorithmic scheduling engine that automatically prevents date conflicts for elephant bookings and verifies total daily inventory availability for rental accessories.
- 3. Role-Based Access:** Design customized dashboard environments for four primary user groups: Admins, Temple Committees, Elephant Owners, and Accessory Rental Owners.
- 4. Verification Gates:** Integrate a vetting workflow where newly listed elephants and accessories remain hidden until an administrator reviews their medical logs and certificates.
- 5. Heritage-Inspired Interface:** Implement a responsive Single Page Application (SPA) with a custom design system incorporating Kerala's traditional colors (Maroon, Gold, Ivory, Green) and typography.

4. SCOPE OF THE PROJECT

The scope of the portal covers all planning and coordination aspects for Kerala temple festivals. The target user roles and boundaries are defined below:

- **Temple Committees:** Register/claim local temples, schedule festival calendars, catalog events, browse verified elephants/accessories, and coordinate booking requests.
- **Elephant/Accessory Owners:** Register assets, specify parameters (age, height, mahouts, stock quantity, rental rates), upload fitness certificates, and accept or decline requests.
- **System Administrator:** Vets document verifications, moderates registrations, and monitors platform usage statistics.

The prototype runs as a client-side Single Page Application (SPA) with a relational database emulated on top of HTML5 localStorage for offline compatibility and demo convenience. Future scopes involve migrating to a REST API backend with PostgreSQL, adding online

payment gateways (escrow support), and integrating real-time GPS tracking for elephant processions.

5. PROPOSED SYSTEM

5.1 System Architecture Overview

The proposed Pooram Connect system is a clean Single Page Application (SPA) utilizing TypeScript. It replaces the manual system with a centralized digital ledger. The system bootstrap maps a custom regex router inside router.ts to intercept all browser clicks, loading and rendering views dynamically without complete page refreshes. The database engine in db.ts emulates relational logic by maintaining schema constraints (Users, Temples, Festivals, Elephants, Accessories, Bookings) on top of localStorage serialization. State adjustments trigger custom DOM events, allowing header and calendar views to update instantly.

5.2 Concurrency and Booking Logic

A critical feature of Pooram Connect is its strict validation rules that prevent conflicts during booking:

1. Elephant Booking Overlap Algorithm: Elephants are treated as single-occupancy resources. When a committee creates a booking request for an elephant, the engine checks for overlaps. For the requested range [StartDate, EndDate], if there exists any booking for the same elephant with status 'confirmed' where $(\text{RequestedStart} \leq \text{BookingEnd}) \text{ AND } (\text{RequestedEnd} \geq \text{BookingStart})$, the transaction is aborted and a calendar exception is thrown. This validation is run during request creation and again during owner confirmation to prevent double bookings from concurrent pending requests.

2. Accessory Stock Verification Algorithm: Traditional accessories are rented in bulk quantities. The engine treats these as multi-quantity inventory assets. When a booking is requested for quantity 'Q' from StartDate to EndDate, the system iterates through each day in the date range. For each date, it sums the quantities of that accessory reserved in all other active, confirmed bookings. The remaining stock is computed as $(\text{QuantityTotal} - \text{AllocatedQuantity})$. If the requested quantity 'Q' exceeds the remaining stock on ANY single day, the system blocks the booking and returns an inventory stock exception.

5.3 Database Relational Schema

The following database schemas define the emulation layer of Pooram Connect. They are mapped with unique IDs acting as Primary Keys (PK) and references acting as Foreign Keys (FK):

Table 5.3.1: Users Schema (users)

Field Name	Data Type	Key	Description
id	String	PK	Unique identifier for the user (u-xxx)
name	String	-	Full name of the stakeholder or authority
email	String	-	Login email (unique value)
passwordHash	String	-	SHA-256 representation of password
role	Enum	-	admin committee elephant_owner accessory_owner
phone	String	-	Mobile contact number
district	String	-	District of operations in Kerala
isVerified	Boolean	-	Flag indicating account status

clearance

Table 5.3.2: Elephants Schema (elephants)

Field Name	Data Type	Key	Description
id	String	PK	Unique identifier for the elephant (e-xxx)
ownerId	String	FK	References users.id (role: elephant_owner)
name	String	-	Majestic name of the elephant (e.g. Karnan)
age	Number	-	Age of the elephant in years
height	Number	-	Height measured in centimeters
weight	Number	-	Weight measured in kilograms
registrationNumber	String	-	Government captive elephant registration ID
fitnessCertificateUrl	String	-	Local URL/Path to veterinary certificate

isVerified	Boolean	-	Status set by admin after profile audit
------------	---------	---	---

Table 5.3.3: Elephant Bookings Schema (elephantBookings)

Field Name	Data Type	Key	Description
id	String	PK	Unique identifier for the booking (eb-xxx)
festivalId	String	FK	References festivals.id
elephantId	String	FK	References elephants.id
startDate	String	-	Booking start date (YYYY-MM-DD)
endDate	String	-	Booking end date (YYYY-MM-DD)
status	Enum	-	pending accepted rejected confirmed cancelled

Table 5.3.4: Accessory Bookings Schema (accessoryBookings)

Field Name	Data Type	Key	Description
id	String	PK	Unique identifier for the accessory

			booking (ab-xxx)
festivalId	String	FK	References festivals.id
accessoryId	String	FK	References accessories.id
startDate	String	-	Rental start date (YYYY-MM-DD)
endDate	String	-	Rental end date (YYYY-MM-DD)
quantity	Number	-	Number of items rented
status	Enum	-	pending accepted rejected confirmed

6. MODULES OF THE PROJECT

Pooram Connect is divided into six core modules, each handling a distinct segment of functionality:

- 1. Auth & Role Module (auth.ts):** Manages user signup, role selection (Committee, Owners, Admins), login, and secure sessions using client-side SHA-256 hashing and sessionStorage.
- 2. Festival Planning Module (festivals.ts):** Allows committees to claim temples, schedule calendars, catalog detailed programs, and dispatch bookings.
- 3. Elephant Directory & Owner Module (elephants.ts):** Enables owners to add physical profiles and certificates, view pending bookings, and accept or reject requests on a calendar.
- 4. Accessory Rental & Inventory Module (accessories.ts):** Enables accessory owners to upload items, input stock count, and manage rental requests based on daily inventory availability.
- 5. Admin Audit & Moderation Module (dashboard.ts):** Provides administrative oversight, listing pending registrations and veterinary certificates to verify them before they go public.
- 6. Routing & UI Engine (router.ts, layout.ts):** Implements a custom regex router to intercept page requests and render views dynamically. Features a responsive heritage-themed design.

7. SOFTWARE AND HARDWARE REQUIREMENTS

The development and execution environments for Pooram Connect are detailed below:

Table 7.1: Software Requirements

Component	Specification
Operating System	Windows 10/11, Linux (Ubuntu), or macOS
IDE / Editors	Visual Studio Code (v1.85+)
Runtime & Package Manager	Node.js (v18.x+) & NPM
Programming Language	TypeScript (v5.x) & ES6+ JavaScript
Bundler / Server	Vite (v5.x) development compiler
Target Browsers	Chrome (v115+), Firefox, Edge, or Safari

Table 7.2: Hardware Requirements

Component	Minimum Specifications
Processor	Intel Core i3 or AMD Ryzen 3 (2.4 GHz minimum)
Memory (RAM)	8 GB DDR4 RAM
Storage System	256 GB SSD (with 10 GB free space)
Display Screen	Standard 15.6-inch screen (1366x768 resolution)

8. EXPECTED OUTCOME

- **No Booking Conflicts:** The overlap algorithm will completely prevent double-booking of elephants.
- **Accessory Rental Tracking:** Rental operators can track inventory availability day-by-day, preventing over-allocations.
- **Enhanced Compliance:** Enforcing veterinary clearances on listings increases public safety at crowded festival venues.
- **Centralized Timelines:** Devotees and tourists gain access to verified schedule timelines, enhancing tourist coordination.

9. CONCLUSION

Pooram Connect successfully digitizes one of the most culturally significant and logistically complex sectors of Kerala: temple festival coordination. By bridging information gaps between committees, elephant owners, and rental services, it adds transparency, security, and administrative efficiency. The Single Page Application (SPA) architecture utilizing TypeScript offers a high-performance experience, while the localized relational database emulates concurrency management. Overall, Pooram Connect acts as a blueprint for blending traditional cultural heritage with modern software engineering, ensuring Kerala's iconic festivals remain safe, transparent, and well-organized.

10. REFERENCES

Below are the primary resources, specifications, and academic guides referenced during the development:

- [1] APJ Abdul Kalam Technological University, Kerala. Regulations for Master of Computer Applications (MCA) Project Work. Academic Syllabus Guidelines, 2024-2026.
- [2] Government of Kerala. Captive Elephant (Management and Keeping) Rules, 2003 & subsequent amendments. Forest and Wildlife Department regulatory reports.
- [3] TypeScript Documentation: Detailed guides on Type Safety, Interfaces, Modules, and Compilation. Available online at: <https://www.typescriptlang.org/docs/>
- [4] Vite Bundling Guide: Next Generation Frontend Tooling. Build performance optimizations. Available online at: <https://vite.dev/guide/>
- [5] MDN Web Docs. client-side storage mechanisms and HTML5 LocalStorage APIs. Available online at: <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>